CS556 ASSIGNMENT REPORT

NAME : BATTULA CHETAN REDDY

ROLL NUMBER : 254156005

DEPARTMENT : MTECH CICPS

Evaluating Network Switch Scheduling Algorithms

A Simulation-Based Comparison of FIFO, VOQ (Optimal), and iSLIP

3 × 3 Crossbar Switch · 18-Packet Input Trace

1. Problem Statement

Modern network switches must forward packets across multiple input and output ports simultaneously without conflict. A **3 × 3 crossbar switch** — with inputs I_0, I_1, I_2 and outputs O_0, O_1, O_2 — operates in synchronous time slots (t = 0, 1, 2, …). In each slot, at most one packet may traverse any given input or output port, so the switch must decide, at every slot, which input-output pairs to connect.

The central challenge is the **scheduling algorithm**: a poor choice causes unnecessary queuing, wasted output-port capacity, and increased latency. This assignment investigates three progressively sophisticated scheduling strategies:

- **Part 1 — FIFO:** Single per-input queue; exposes Head-of-Line (HoL) blocking.
- **Part 2 — VOQ with Optimal Matching:** Separate queue per (input, output) pair; exhaustive bipartite matching guarantees minimum service time.
- **Part 3 — iSLIP:** VOQ architecture with practical round-robin arbitration in Request–Grant–Accept phases.

All three algorithms are evaluated on the same 18-packet input trace, allowing a direct, controlled comparison of throughput, backlog, HoL events, and total service time.

2. System Model & Input Trace

2.1 Switch Fabric

- **Topology:** 3 × 3 crossbar; inputs I_0–I_2, outputs O_0–O_2.
- **Time slots:** Synchronous, t = 0, 1, 2, … Each port handles exactly one packet per slot.
- **Propagation:** Processing and switching each take exactly 1 time slot.
- **Tie-breaking (FIFO):** If multiple inputs contend for the same output, the lowest-numbered input wins (I_0 beats I_1 beats I_2).

2.2 Input Trace

The following 18-packet trace is used for all three simulations. Packets arrive at the **beginning** of their designated time slot.

Packet ID	Arrival (t)	Source Input	Destination Output
p1	0	I0	O0
p2	0	I0	O1

p3	0	I1	O0
p4	0	I1	O2
p5	0	I2	O0
p6	1	I0	O2
p7	1	I2	O1
p8	2	I1	O1
p9	2	I2	O2
p10	3	I0	O1
p11	3	I1	O0
p12	3	I2	O1
p13	4	I0	O0
p14	4	I1	O2
p15	4	I2	O2
p16	5	I0	O2
p17	5	I1	O1
p18	5	I2	O0

3. Theory — Head-of-Line Blocking & Virtual Output Queuing

3.1 Head-of-Line (HoL) Blocking

In a standard input-queued switch with a single FIFO per input port, only the **head packet** of each queue is eligible for transmission in any given slot. If that head packet's destination output port is already busy serving another input, the entire queue behind it is frozen — even if packets further back have free destination ports. This is **Head-of-Line blocking.**

Consider t = 0 in this trace. Input I_0 holds p1 ($\rightarrow O_0$), p2 ($\rightarrow O_1$), p6 ($\rightarrow O_2$). Input I_1 holds p3 ($\rightarrow O_0$) and p4 ($\rightarrow O_1 \rightarrow O_2$). Input I_2 holds p5 ($\rightarrow O_0$). The head of I_1 is **p3 ($\rightarrow O_0$)**, which conflicts with p1 already being sent on O_0. So p3 is blocked, and critically, **p4 ($\rightarrow O_2$)** — which has a free output — is also blocked behind p3. The same pattern traps **p5 ($\rightarrow O_0$)** at the head of I_2. This single slot sees only 1 packet sent instead of a possible 3, wasting 66% of capacity.

3.2 Virtual Output Queuing (VOQ)

VOQ eliminates HoL blocking by maintaining **N separate queues per input** — one for each destination output. For a 3 × 3 switch, each input has 3 VOQs: VOQ[i][0], VOQ[i][1], VOQ[i][2]. A packet is never blocked by a packet heading to a different output.

The scheduling problem becomes finding a **valid bipartite matching** between inputs and outputs: no two inputs send to the same output, and no output receives from more than one input. The maximum possible matching is N = 3 packets per slot for a 3 × 3 switch.

3.3 Optimal Matching (VOQ — Part 2)

The Part 2 implementation enumerates **all N! = 6 permutations** of output assignments at every time slot and selects the permutation that maximises the number of active (non-empty) VOQ

pairings. This brute-force approach is guaranteed to find the theoretical minimum service time but is computationally infeasible for large N.

4. Theory — iSLIP Scheduling

4.1 Overview

iSLIP (iterative SLIP) replaces the exponential exhaustive search with an efficient **round-robin arbitration** scheme. Each slot executes one or more iterations of a three-phase protocol. This assignment implements a **single-iteration** iSLIP per slot.

4.2 The Three Phases

Phase 1 — Request

Each input port sends a **request** to every output port for which it has at least one packet waiting in its VOQ. Multiple outputs can be requested simultaneously.

Phase 2 — Grant

Each output port examines its list of requesting inputs and **grants** exactly one of them — the one pointed to by its round-robin pointer (scanning from the pointer position modulo N). The pointer's current position is the starting input index for that scan.

Phase 3 — Accept

Each input port may receive grants from multiple outputs. It **accepts exactly one** — again using its own round-robin pointer to scan from its current position. Once it accepts a grant, it dequeues the head packet from that VOQ, and both its pointer and the granting output's pointer advance to **(accepted/granted index + 1) mod N**. Pointers for unmatched arbiters do not advance.

4.3 Significance of Desynchronised Pointers

The round-robin pointers begin at 0 but evolve independently as each arbiter advances only when matched. Over time, different arbiters reach different pointer positions, **breaking the initial symmetry**. This desynchronisation is crucial: it prevents repeated starvation of the same input or output in the presence of hot spots, and ensures long-run fairness across all (input, output) pairs. Without pointer advancement, every slot would repeat the same priority order and systematically under-serve the last input or output in the round-robin.

5. Part 1 — FIFO Simulation Results

5.1 Algorithm Design

Each input port maintains a single FIFO queue. On every time slot: (1) Arriving packets are enqueued. (2) For each input in order $I_0 \rightarrow I_1 \rightarrow I_2$, the head packet is forwarded if its destination output is not already in use this slot. (3) If the head packet's output is busy, the entire input queue is stalled — HoL blocking is recorded.

5.2 Slot-by-Slot Execution Log

Slot (t)	Packets Sent	HoL Blocked	Backlog
t = 0	p1 (I0→O0)	p3 (I1→O0), p5 (I2→O0)	4
t = 1	p2 (I0→O1), p3 (I1→O0)	p5 (I2→O0)	4
t = 2	p6 (I0→O2), p5 (I2→O0)	p4 (I1→O2)	4
t = 3	p10 (I0→O1), p4 (I1→O2)	p7 (I2→O1)	5
t = 4	p13 (I0→O0), p8 (I1→O1)	p7 (I2→O1)	6
t = 5	p16 (I0→O2), p11 (I1→O0), p7 (I2→O1)	—	6
t = 6	p14 (I1→O2)	p9 (I2→O2)	5
t = 7	p17 (I1→O1), p9 (I2→O2)	—	3
t = 8	p12 (I2→O1)	—	2
t = 9	p15 (I2→O2)	—	1
t = 10	p18 (I2→O0)	—	0

5.3 Key Observations

- **t = 0 — Two simultaneous HoL blocks:** p3 (I1→O0) is blocked because p1 already claimed O0. p5 (I2→O0) is also blocked for the same reason. Only p1 is sent — 1 packet out of a possible 3.
- **t = 1 — Cascading delay:** p5 (→O0) remains blocked at the head of I2 because p3 now takes O0. This forces p5 to wait until t=2.
- **t = 3 — p7 trapped:** p7 (I2→O1) arrives at t=1 but p7 blocks behind p5 in the I2 queue until t=5 because p5 keeps the head of I2 occupied while O0 is busy.
- **t = 4 — Peak-backlog onset:** New packets arrive (p13, p14, p15) while the backlog is still 6, showing how FIFO cannot drain fast enough to absorb bursts.
- **Total service time: t = 10** (11 slots). Seven HoL blocking events reduce effective throughput to 1.64 packets/slot — only 54.5% link utilisation.

6. Part 2 — VOQ with Optimal Matching

6.1 Algorithm Design

Each input port has 3 separate VOQs — VOQ[i][o] for destination o. On every slot, the scheduler enumerates all 6 permutations of output assignments and selects the one that matches the maximum number of non-empty VOQs, maximising per-slot throughput. Packets within each VOQ are served FIFO.

6.2 Slot-by-Slot Execution Log

Slot (t)	Packets Sent (Max Matching)	Backlog
t = 0	p2 (I0→O1), p4 (I1→O2), p5 (I2→O0)	2
t = 1	p6 (I0→O2), p3 (I1→O0), p7 (I2→O1)	1
t = 2	p1 (I0→O0), p8 (I1→O1), p9 (I2→O2)	0
t = 3	p10 (I0→O1), p11 (I1→O0)	1
t = 4	p13 (I0→O0), p14 (I1→O2), p12 (I2→O1)	1
t = 5	p16 (I0→O2), p17 (I1→O1), p18 (I2→O0)	1
t = 6	p15 (I2→O2)	0

6.3 Key Observations

- **t = 0 — Immediate 3-way parallelism:** p2 (I0→O1), p4 (I1→O2), p5 (I2→O0) form a conflict-free matching. All three outputs are used simultaneously. VOQ allows p4 to be sent even though p3 (→O0) is also queued at I1 — no HoL block occurs.
- **t = 1,2 — Sustained full throughput:** 3 packets sent per slot in each of the first three time slots, clearing 9 packets in 3 slots.
- **t = 3 — Only 2 packets:** Only p10 and p11 arrive and no earlier backlog remains at this point, so the maximum matching has size 2.
- **Zero HoL blocking:** VOQs completely eliminate head-of-line blocking. Every queue can be independently served.
- **Total service time: t = 6**. Average throughput 2.57 packets/slot — 85.7% link utilisation. Best possible for this trace.

7. Part 3 — iSLIP Simulation Results

7.1 Algorithm Design

The VOQ architecture from Part 2 is retained. A single iSLIP iteration per slot is performed: inputs request all outputs with pending VOQ traffic; outputs grant one input each via round-robin starting from their current pointer; inputs accept one grant each via their own round-robin pointer. Both pointer sets update only when a match is made.

7.2 Slot-by-Slot Execution Log

Slot (t)	Packets Sent	Arbiter Pointers	Backlog
t = 0	p1 (I0→O0), p4 (I1→O2)	In:[1,3,1] Out:[1,2,1]	3
t = 1	p2 (I0→O1), p3 (I1→O0)	In:[2,1,1] Out:[2,2,1]	3
t = 2	p8 (I1→O1), p5 (I2→O0)	In:[1,2,1] Out:[1,2,1]	3
t = 3	p11 (I1→O0), p7 (I2→O1)	In:[2,1,2] Out:[2,1,2]	4
t = 4	p13 (I0→O0), p9 (I2→O2)	In:[1,1,0] Out:[1,2,0]	5
t = 5	p10 (I0→O1), p18 (I2→O0)	In:[2,1,1] Out:[2,2,1]	6
t = 6	p6 (I0→O2), p17 (I1→O1)	In:[0,2,1] Out:[0,2,1]	4
t = 7	p14 (I1→O2), p12 (I2→O1)	In:[1,0,2] Out:[1,0,2]	2

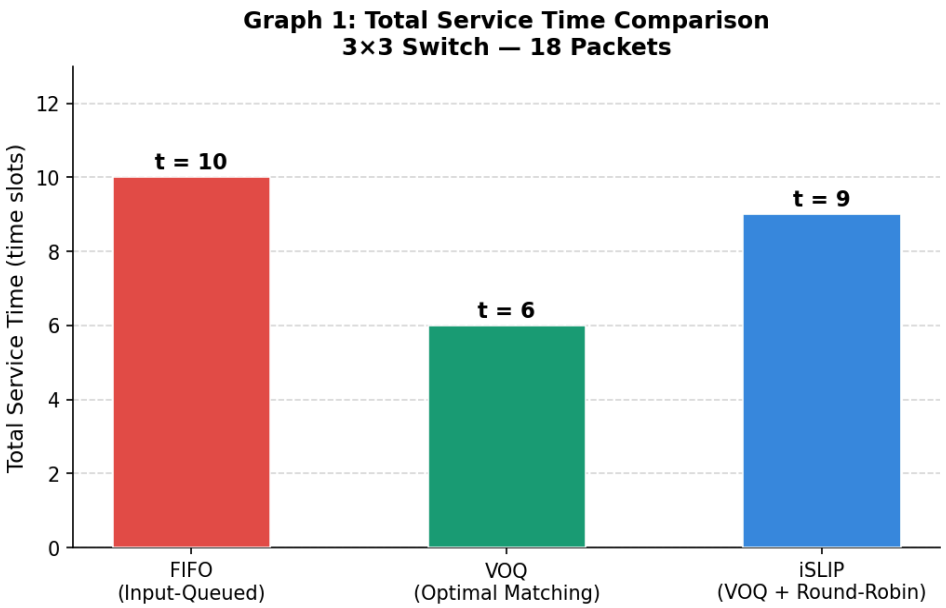
t = 8	p15 (I2→O2)	In:[1,0,0] Out:[1,0,0]	1
t = 9	p16 (I0→O2)	In:[0,0,0] Out:[0,0,0]	0

7.3 Key Observations

- **t = 0 — Pointer initialisation:** All pointers start at 0. I0's pointer at 0 means it accepts the first available grant in round-robin order. O0 grants I0 (pointer 0 → scans for I0 first). This produces the matching {I0→O0, I1→O2, I2→?} — only 2 packets vs VOQ's 3, because I2 has no grant accepted at t=0.
- **Steady 2 packets/slot:** iSLIP delivers a consistent 2 packets/slot across t=0 to t=7. This reflects the sub-optimality of single-iteration iSLIP compared to optimal matching — some conflicts resolve in later iterations that are omitted here.
- **No HoL blocking:** VOQ architecture ensures all queue heads are independently eligible, just as in Part 2.
- **Pointer evolution drives fairness:** By t=3, the input pointers are [2, 1, 2] and output pointers are [2, 1, 2], showing desynchronisation. This prevents any single (input, output) pair from being perpetually deprioritised.
- **Total service time: t = 9**. Average throughput 1.80 packets/slot — 60.0% utilisation. 3 slots slower than optimal VOQ but 1 slot faster than FIFO.

8. Data Visualization & Analysis

8.1 Graph 1 — Total Service Time Comparison

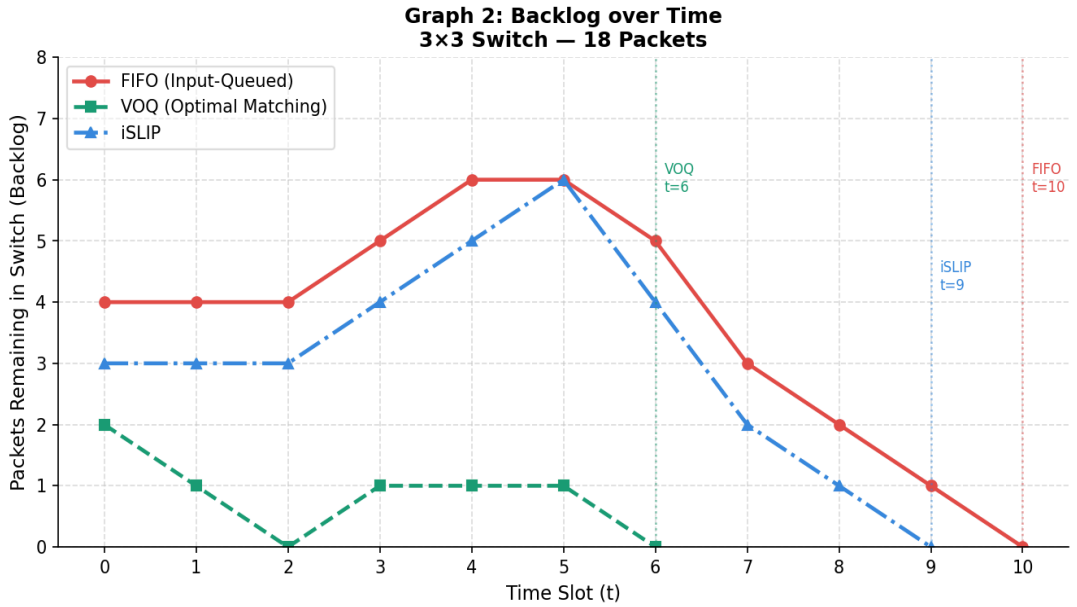


Graph 1: Bar chart comparing total service time for FIFO (t=10), VOQ Optimal (t=6), and iSLIP (t=9)

Analysis: VOQ with optimal matching completes all 18 packets 4 slots earlier than FIFO and 3 slots earlier than iSLIP. The difference between iSLIP and optimal (3 slots) arises because single-iteration iSLIP sometimes fails to maximise the matching in one pass — a second round of

Request–Grant–Accept would improve this. FIFO's additional penalty comes entirely from HoL blocking.

8.2 Graph 2 — Backlog over Time



Graph 2: Line chart showing packets remaining in switch vs time slot for FIFO, VOQ Optimal, and iSLIP

Analysis — FIFO (red): The FIFO backlog grows from 4 at t=0 to a peak of 6 at t=4–5, driven by new packet arrivals at t=3, 4, 5 that arrive faster than FIFO can drain them. HoL blocking at t=0 (only 1 packet sent) sets the stage for this accumulation. The backlog does not reach zero until t=10.

Analysis — VOQ Optimal (green): Backlog is immediately low (2 at t=0), briefly touches 0 at t=2, and never exceeds 2. The exhaustive matching sends 3 packets/slot whenever possible, fully absorbing all arrivals and draining the switch by t=6.

Analysis — iSLIP (blue): iSLIP's trajectory mirrors FIFO's in shape — it also peaks at 6 around t=5 — but recovers faster, completing at t=9. The identical peak backlog to FIFO (both at 6) shows that the burst at t=3–5 overwhelms both schedulers; the difference is that iSLIP's 2-packet/slot rate without HoL blocking lets it recover more efficiently.

9. Comparative Performance Summary

Metric	FIFO	VOQ (Optimal)	iSLIP
Completion time (t)	t = 10	t = 6	t = 9
Avg. throughput (pkt/slot)	1.64	2.57	1.80
Max backlog	6	2	6
Avg. backlog	3.64	0.86	3.10
HoL blocking events	7	0	0
Switch link utilisation	54.5%	85.7%	60.0%

9.1 Why FIFO Performs Worst

FIFO's fundamental problem is that a single queue per input serialises all packets regardless of destination. The 7 HoL blocking events collectively waste approximately 7–10 packet-forwarding opportunities. Consider t=0: three output ports are idle but only 1 packet is sent. FIFO's link utilisation of 54.5% means nearly half the switch capacity is squandered on HoL stalls.

9.2 Why VOQ Optimal Performs Best

By isolating each (input, output) flow into its own queue, VOQ completely removes the head-of-line interference. The exhaustive matching then guarantees that the maximum number of non-conflicting packets is forwarded every slot. This produces 85.7% link utilisation — the maximum achievable for this particular traffic pattern (the 14.3% gap reflects time slots where insufficient queued traffic prevents a full 3-way match).

9.3 iSLIP as a Practical Compromise

iSLIP retains VOQ's freedom from HoL blocking while replacing the O(N!) exhaustive search with an O(N) round-robin arbitration. Single-iteration iSLIP trades 3 extra time slots (vs optimal) for a polynomial-time algorithm suitable for hardware implementation. In real switches (e.g., with N = 128), the O(N!) approach is completely intractable; iSLIP with multiple iterations converges to near-optimal matching in O(N) iterations per slot.

10. Conclusion

This assignment demonstrated through simulation that queuing architecture and scheduling algorithm selection have a profound impact on switch performance.

- **FIFO** is architecturally simple but suffers severe HoL blocking, achieving only 54.5% link utilisation and the longest service time (t=10).
- **VOQ with optimal matching** eliminates HoL blocking and achieves the theoretical best service time (t=6) at 85.7% utilisation, at the cost of exponential computation.
- **iSLIP** bridges the gap: it eliminates HoL blocking, runs in polynomial time, achieves 60% utilisation, and completes at t=9 — only 3 slots behind optimal.

For real-world high-speed routers and data-centre switches, iSLIP (or multi-iteration variants) represents the standard practical choice. The desynchronised round-robin pointers ensure long-run fairness across all flows, while the VOQ architecture guarantees that no flow starves another through head-of-line interference.

Simulation implemented in C++ (performance.cpp, performance2.cpp, performance3.cpp). Data analysis in Python (scheduling_analysis.py).